

CH10 多实例架构

考试复习笔记

来源：开源操作系统课件（47 页） 日期：2026 年 6 月 23 日

1 单实例的瓶颈 [p.5–8]

1.1 资源瓶颈 [p.7]

资源	瓶颈表现
CPU	大量计算、JSON 序列化、模板渲染、加解密
内存	频繁 GC、Swap、OOM、进程被杀
线程池/事件循环	工作线程都在等待外部服务，新请求排队
数据库连接池	所有连接被占用，新请求等待
文件描述符	高并发连接消耗打开文件数量（socket = fd）
网络带宽	响应体大（图片/视频/大文件下载）
磁盘 I/O	日志写入、文件上传、Redis 持久化

1.2 单点故障风险 [p.8]

单实例部署 = 单点故障 (SPOF)：所有用户无法访问、交易中断、数据不一致、恢复时间 = 重启 + 数据检查。

2 扩展的基本概念 [p.9–15]

2.1 垂直扩展 vs 水平扩展 [p.9–10]

维度	垂直扩展（Scale Up）	水平扩展（Scale Out）
方式	提升单机配置（CPU/内存/磁盘）	增加服务器数量
架构复杂度	低	高
成本曲线	指数级（高端硬件溢价）	线性
扩展上限	有物理上限	理论上无上限
高可用	单点故障	天然支持
适用场景	初期、数据密集型	高并发、高可用需求

2.2 高并发性能指标 [p.11–15]

指标	定义
QPS (Queries Per Second)	每秒查询数，适用于只读场景
TPS (Transactions Per Second)	每秒事务数（一个事务可能含多个请求），适用于写场景
并发连接数	当前同时保持的 TCP 连接数。 并发连接数 QPS
吞吐量 (Throughput)	单位时间处理的请求总数 (QPS/TPS)
响应时间	P50/P90/P99 分位数。P99 飙升 = 性能劣化信号

2.3 可用性等级 [p.14]

可用性	年停机时间
99% (基本可用)	3.65 天
99.9% (较高可用)	8.76 小时
99.99% (高可用)	52.6 分钟
99.999% (极高可用)	5.26 分钟

SLA (Service Level Agreement): 服务等级协议，约定可用性承诺、计算方式、违约赔偿、免责条款。

3 多实例带来的问题 [p.21–38]

3.1 六大核心问题 [p.21]

1. 请求如何分发？ → 负载均衡
2. 坏实例如何发现？ → 健康检查
3. **Session** 在哪里？ → Session 外置 (Redis) 或无状态认证 (JWT)
4. 上传文件在哪里？ → 共享存储/对象存储 (OSS/S3/MinIO)
5. 日志如何查看？ → 请求 ID + 集中日志机制
6. 发布如何进行？ → 滚动发布 + 优雅下线

3.2 负载均衡 [p.22–26]

同 CH07: 四层 (L4, IP+Port) vs 七层 (L7, HTTP 内容)。策略: RR/权重/IP 哈希/fair/URL 哈希。

3.3 健康检查 [p.27–30]

主动健康检查: 负载均衡器定期访问 /health，连续 N 次失败摘除，连续 M 次成功恢复。

被动失败检测: 基于真实请求结果判断（连接失败/5xx/超时）。

3.4 无状态应用 [p.31–32]

有状态问题：Session 存在实例内存 → 分配到其他实例 → Session 丢失 → 重新登录。

无状态方案：Session 存 Redis（共享存储）→ 任意实例都能获取 → 无状态应用。

3.5 文件上传外置 [p.33–34]

本地存储问题：文件存在实例 1 → 请求分配到实例 2 → 文件不存在。

解决方案：对象存储（OSS/S3/MinIO），所有实例通过 URL 访问同一份文件。

4 高可用架构 [p.40–46]

4.1 三个核心思想 [p.40]

1. 冗余：部署多份资源，一份故障时其他副本接管
2. 故障隔离：防止局部故障扩散到整个系统
3. 故障转移：主节点故障时自动切换到备用节点

4.2 多层次冗余设计 [p.41]

- 应用层：多实例 + Nginx 负载均衡；Nginx 主备 (Keepalived/VRRP)
- 数据层：MySQL 主从复制；Redis 主从 + Sentinel
- 基础设施：多可用区（同城双活）、多地域（异地灾备）

4.3 故障隔离策略 [p.42]

资源隔离（不同业务不同线程池）→ 进程隔离（不同服务独立进程）→ 网络隔离（内外网分离）→ 熔断隔离（下游故障时快速失败）。

4.4 故障转移场景 [p.43]

1. 应用实例故障：Nginx 健康检查发现 → 自动摘除 → 恢复后自动加入
2. Nginx 主节点故障：Keepalived 检测 → VIP 漂移到备节点 → 用户无感知
3. Redis 主节点故障：Sentinel 检测 → 自动将从提升为主 → 应用重连新主

4.5 服务降级与恢复 [p.44–45]

服务降级：压力过大或部分故障时，主动牺牲非核心功能，保障核心功能可用。

RTO (Recovery Time Objective)：恢复时间目标，从故障到恢复的最大可接受时间。

RPO (Recovery Point Objective)：恢复点目标，可接受的最大数据丢失量。

PetStore 示例：核心交易 $RTO < 5\text{min}$ ， $RPO = 0$ （不能丢数据）。

开源操作系统 · CH10 多实例架构 · 47 页课件全覆盖